Experiment No: 1

Aim: Design and Implement a python program which demonstrates intelligent behaviour using simple rules.

Theory:

You can design an intelligent Python program using simple rules by creating a **Rule-Based System** that uses `if-elif-else` statements and functions to process inputs and make decisions based on predefined conditions, demonstrating behavior like pattern matching, logical inference, and basic decision-making without complex machine learning. A great example is an agent that follows a "think, plan, and act" loop, using rules to determine the best response to user queries.

Problem Statement

Develop a program that decides whether a student is **Eligible or Not Eligible for an exam** based on attendance percentage.

Rules

If attendance ≥ 75% → **Eligible for Exam**

If attendance < 75% → **Not Eligible for Exam**

Algorithm

1. Start
2. Input attendance percentage
3. Check condition using if–else
4. Display decision
5. Stop

Python Code

```
# Intelligent behavior using simple rules
attendance = float(input("Enter attendance percentage: "))
if attendance >= 75:
print("Student is Eligible for Exam")
```

```
else:
print("Student is Not Eligible for Exam")
```

Sample Output

Enter attendance percentage: 82

Student is Eligible for Exam

Conclusion

This program demonstrates a **basic AI rule-based system**, where decisions are made using predefined rules without learning from data.

Problem Statement

Develop a simple **medical expert system** that suggests a possible illness based on symptoms.

Knowledge Base (Rules)

If **fever** and **cough** → Flu

If **fever** and **headache** → Viral Infection

If **stomach pain** → Food Poisoning

Else → Consult a Doctor

Algorithm

1. Start
2. Input symptoms from user
3. Match symptoms with rules
4. Display diagnosis
5. Stop

Python Code

```
# Rule-Based Expert System
symptom1 = input("Enter first symptom: ").lower()
symptom2 = input("Enter second symptom (or none): ").lower()
if symptom1 == "fever" and symptom2 == "cough":
print("Possible Diagnosis: Flu")
```

```
elif symptom1 == "fever" and symptom2 == "headache":
print("Possible Diagnosis: Viral Infection")
elif symptom1 == "stomach pain" or symptom2 == "stomach pain":
print("Possible Diagnosis: Food Poisoning")
else:
print("Diagnosis: Consult a Doctor")
```

Sample Output

Enter first symptom: fever

Enter second symptom (or none): cough

Possible Diagnosis: Flu

Conclusion

- This practical demonstrates a **rule-based expert system** where decisions are made using predefined knowledge and inference rules.

Experiment No: 2

Aim: Design a simple chatbot can be built using basic Python fundamentals such as loops, conditional statements, and string matching.

Theory:

A simple chatbot can be designed using Python's fundamental building blocks, primarily loops, conditional statements, and basic string operations for input/output and matching. This structure allows the program to process user input and provide predefined responses.

Algorithm:

- The basic algorithm looks like this:
- Define Responses: Store common greetings, questions, and responses in a way the program can easily access them (e.g., using a dictionary or simple lists of keywords).
- Start Loop: Use a while loop that continues indefinitely until a specific exit command is given.
- Get Input: Prompt the user for input using the input() function. Convert the input to lowercase using .lower() for easier matching.
- Process Input (String Matching): Use if/elif statements and string methods like in or == to check if keywords or specific phrases are present in the user's input.
- Provide Output: Print a predefined response based on which condition was met.
- Handle Unknown Input: Include an else block to provide a generic response if no keywords are matched.
- Exit Condition: Include a check for commands like "bye" or "quit" to break the loop, ending the program.

Coding:

```
import random
import re

def simple_chatbot():
    """A basic chatbot using fundamental Python concepts."""
    # 1. Define responses in a dictionary (key: user input keywords, value: bot response)
    # Using lowercase for easier matching
    responses = {
        "hello": ["Hi there!", "Hello!", "Greetings!"],
        "hi": ["Hi there!", "Hello!", "Greetings!"],
        "how are you": ["I'm a bot, I'm doing great!", "I'm functioning optimally."],
        "what is your name": ["You can call me PyBot.", "I don't have a name, but I respond to 'Hey
you!'"],
        "bye": ["Goodbye!", "See you later!", "Have a great day!"],
        "exit": ["Goodbye!", "See you later!", "Have a great day!"]
    }
    # Default responses for when no match is found
    default_responses = [
        "Sorry, I didn't understand that.",
        "Could you please rephrase that?",
        "I'm not sure how to respond to that."
    ]
    print("PyBot: Hi! I'm a simple chatbot. Type 'bye' or 'exit' to end the conversation.")
    # 2. Main conversation loop
    while True:
        user_input = input("You: ").lower() # Convert input to lowercase for case-insensitive matching
        # 3. Simple input cleaning
        # Remove punctuation to improve matching
        user_input = re.sub(r'[^\w\s]', '', user_input)
            # 4. Check for exit conditions
```

```
if user_input in ["bye", "exit"]:
        print(f"PyBot: {random.choice(responses[user_input])}")
        break # Exit the while loop to end the program


    # 5. Matching logic using loops and conditionals
    matched = False
    for key in responses:
        # Check if any keyword in the dictionary key is in the user's input
        if key in user_input:
            # Select a random response from the list of options
            bot_response = random.choice(responses[key])
            print(f"PyBot: {bot_response}")
            matched = True
            break # Break the loop once a match is found
    # 6. Default response if no match was found
    if not matched:
        print(f"PyBot: {random.choice(default_responses)}")
# Run the chatbot
if __name__ == "__main__":
    simple_chatbot()
```

Sample Output:

PyBot: Hi! I'm a simple chatbot. Type 'bye' or 'exit' to end the conversation.

You: Hello

PyBot: Hi there!

You: how are you

PyBot: I'm a bot, I'm doing great!

You: what is your name

PyBot: I don't have a name, but I respond to 'Hey you!'

You: abc

PyBot: Sorry, I didn't understand that.

You: @@@

PyBot: Could you please rephrase that?

You: HELLO^%&

PyBot: Hi there!

You: Bye

PyBot: See you later!

Conclusion:

In conclusion, a simple chatbot can be effectively designed using basic Python fundamentals such as loops, conditional statements, and string matching. Even without advanced libraries or artificial intelligence techniques, the chatbot can interact with users, understand simple inputs, and provide appropriate responses. This approach helps beginners understand core programming concepts like control flow, input handling, and decision-making. Building such a chatbot is a good starting point for learning Python and forms a foundation for developing more advanced chat bots using natural language processing and machine learning in the future.

Experiment No: 3

Aim: Write a python script defines a function is_tautology that checks if a given expression is a tautology.

Theory:

This Python script defines a function `is_tautology` that evaluates whether a given propositional logic expression is a tautology (always True) by generating a truth table for all possible truth assignments of its variables

Coding:

```
import itertools
def is_tautology(expression):
    """
    Checks if a given propositional logic expression is a tautology.
    Variables in the expression must be lowercase letters (e.g., 'p', 'q').
    Logical operators supported: ' and ', ' or ', ' not ', ' == ' (for iff), ' != ' (for xor).
        Example:
        is_tautology("(p or not p)") -> True
        is_tautology("(p and q)") -> False
    """
    # 1. Identify unique variables (lowercase letters)
    variables = sorted(list(set(c for c in expression if 'a' <= c <= 'z')))
      if not variables:
        # If no variables, just evaluate the constant expression
        return bool(eval(expression))
    # 2. Generate all truth assignments (2^n combinations)
    num_vars = len(variables)
    truth_assignments = list(itertools.product([True, False], repeat=num_vars))
    results = []
    # 3. Evaluate the expression for each assignment
    for assignment in truth_assignments:
```

```
    env = dict(zip(variables, assignment))
        try:
            # Use eval to compute the truth value
            result = eval(expression, {}, env)
            results.append(result)
        except Exception as e:
            print(f"Error evaluating '{expression}': {e}")
            return False
    # 4. A tautology is true for all possible assignments
    return all(results)

# --- Test Cases ---
if __name__ == "__main__":
    # Test 1: P or not P (Classic Tautology)
    expr1 = "(p or not p)"
    print(f"Is '{expr1}' a tautology? {is_tautology(expr1)}")

    # Test 2: P and not P (Contradiction)
    expr2 = "(p and not p)"
    print(f"Is '{expr2}' a tautology? {is_tautology(expr2)}")

    # Test 3: Complex Expression
    expr3 = "((p or q) and not p) <= q" # (P or Q) and not P implies Q
    print(f"Is '{expr3}' a tautology? {is_tautology(expr3)}")



    # Test 4: (p implies q) or (q implies p)
    # Using pythonic syntax: (not p or q) or (not q or p)
    expr4 = "(not p or q) or (not q or p)"
    print(f"Is '{expr4}' a tautology? {is_tautology(expr4)}")
```

Output:

Is '(p or not p)' a tautology? True

Is '(p and not p)' a tautology? False

Is '((p or q) and not p) <= q' a tautology? True

Is '(not p or q) or (not q or p)' a tautology? True

Conclusion:

The is_tautology function demonstrates how fundamental Python concepts such as functions, loops, conditional statements, and basic string handling can be used to solve logical problems. By systematically evaluating a logical expression for all possible truth value combinations, the script accurately determines whether the expression is a tautology. This approach strengthens understanding of propositional logic as well as core programming skills, and it serves as a foundation for more advanced logical reasoning and automated theorem-checking applications.

Experiment No: 4

Aim: Implement knowledge representation and logical inference for decision-making in the Wumpus World environment.

Theory:

The Wumpus World is a classic example of a knowledge-based agent in AI. It involves reasoning, knowledge representation and planning. In this agent uses its knowledge of the world to make decisions and navigate safely in the given environment.

Elements:

- Pits: If the agent steps into a pit it falls and dies. A breeze in adjacent rooms suggests nearby pits.
- Wumpus: A creature that kills agent if it enters its room. Rooms next to the Wumpus have a stench. Agent can use an arrow to kill the Wumpus.
- Treasure: Agent's main objective is to collect the treasure (gold) which is located in one room.
- Breeze: Indicates a pit is nearby.
- Stench: Indicates the Wumpus is nearby.

Coding:

//Employee.java

```
# Simple Wumpus World Implementation
# ---------------------------------
# Grid size: 4x4
# P = Pit, W = Wumpus, G = Gold

GRID = 4
# Environment (fixed)
wumpus = (1, 2)
pit = (2, 1)
gold = (3, 3)
# Agent knowledge
safe_cells = set()
visited = set()
```

```
possible_pit = set()
possible_wumpus = set()

# Start position
agent_pos = (0, 0)
safe_cells.add(agent_pos)

def neighbors(cell):
    x, y = cell
    n = []
    if x > 0: n.append((x-1, y))
    if x < GRID-1: n.append((x+1, y))
    if y > 0: n.append((x, y-1))
    if y < GRID-1: n.append((x, y+1))
    return n

def perceive(cell):
    percepts = []
    for n in neighbors(cell):
        if n == pit:
            percepts.append("Breeze")
        if n == wumpus:
            percepts.append("Stench")
    return percepts

def infer(cell, percepts):
    adj = neighbors(cell)
    if "Breeze" not in percepts:
        for a in adj:
            safe_cells.add(a)
    else:
```

```
    for a in adj:
            possible_pit.add(a)



    if "Stench" not in percepts:
        for a in adj:
            safe_cells.add(a)
    else:
        for a in adj:
            possible_wumpus.add(a)

def choose_move(cell):
    for n in neighbors(cell):
        if n in safe_cells and n not in visited:
            return n
    return None

# ---------------- MAIN LOOP ----------------

print("Simple Wumpus World Simulation\n")

for step in range(10):
    print(f"Step {step+1}")
    print("Agent at:", agent_pos)

    visited.add(agent_pos)
    percepts = perceive(agent_pos)
    print("Percepts:", percepts if percepts else "None")

    infer(agent_pos, percepts)
```

```
    move = choose_move(agent_pos)
    if move:
        print("Moving to:", move, "\n")
        agent_pos = move
    else:
        print("No safe move available. Stopping.\n")
        break


print("Visited cells:", visited)
print("Safe cells:", safe_cells)
print("Possible pits:", possible_pit)
print("Possible wumpus:", possible_wumpus)

if agent_pos == gold:
    print("Agent found the gold!")
else:
    print("Agent did not reach the gold.")
```

Output:

```
Simple Wumpus World Simulation

Step 1
Agent at: (0, 0)
Percepts: None
Moving to: (1, 0)

Step 2
Agent at: (1, 0)
Percepts: None
Moving to: (2, 0)
```

Step 3

Agent at: (2, 0)

Percepts: ['Breeze']

Moving to: (3, 0)

Step 4

Agent at: (3, 0)

Percepts: None

Moving to: (3, 1)

Step 5

Agent at: (3, 1)

Percepts: ['Breeze']

Moving to: (2, 1)

Step 6

Agent at: (2, 1)

Percepts: None

Moving to: (1, 1)

Step 7

Agent at: (1, 1)

Percepts: ['Breeze', 'Stench']

Moving to: (0, 1)

Step 8

Agent at: (0, 1)

Percepts: None

Moving to: (0, 2)

Step 9

Agent at: (0, 2)

Percepts: ['Stench']

Moving to: (1, 2)

Step 10

Agent at: (1, 2)

Percepts: None

Moving to: (2, 2)

Visited cells: {(0, 1), (1, 2), (2, 1), (0, 0), (3, 1), (1, 1), (2, 0), (3, 0), (0, 2), (1, 0)}

Safe cells: {(0, 1), (1, 2), (2, 1), (0, 0), (3, 1), (1, 1), (0, 3), (2, 0), (3, 0), (0, 2), (2, 2), (1, 0), (3, 2), (1, 3)}

Possible pits: {(0, 1), (1, 2), (2, 1), (3, 0), (1, 0), (3, 2)}

Possible wumpus: {(0, 1), (1, 2), (2, 1), (0, 3), (1, 0)}

Agent did not reach the gold.

Conclusion:

The Wumpus World environment effectively demonstrates how an intelligent agent can use **knowledge representation and logical inference** to make rational decisions in an uncertain world. By representing percepts such as breeze and stench using logical symbols and rules, the agent is able to reason about the presence or absence of hazards like pits and the Wumpus. Logical inference allows the agent to derive new knowledge from observed percepts and update its knowledge base dynamically.

Based on the inferred knowledge, the agent makes informed decisions by choosing only those actions that lead to safe cells, thereby minimizing risk and improving performance. This approach highlights the importance of reasoning over mere reaction and shows how perception, knowledge, and inference work together to guide intelligent behavior. Overall, the Wumpus World serves as a fundamental model for understanding logic-based decision-making and reasoning in artificial intelligence systems.

Experiment No: 5

Aim: Design a semantic network that represents objects, concepts, and their interrelationships for effective knowledge representation in artificial intelligence using python libraries.

Theory:

To design a semantic network in Python, you can utilize libraries such as NetworkX for graph structure and analysis, and NLTK/spaCy for natural language processing (NLP) tasks to extract relevant entities and relationships from text.

Design and implementation steps:

- Define Entities and Relationships: Identify the core objects (nodes) and the specific relationships (edges) you want to represent (e.g., "is-a," "has-a," "located-in").
- Extract Knowledge: Use NLP libraries like spaCy to automatically identify entities and relationships from a corpus of text, or manually input facts as triples (subject, relationship, object).
- Build the Graph: Use the NetworkX library to programmatically create a directed graph, adding nodes and edges with relevant attributes (e.g., `type='person'`, `relationship='is-a'`).
- Analyze and Visualize: Apply graph algorithms (e.g., centrality measures) and visualize the network using NetworkX's drawing tools or interactive libraries like Pyvis.
- Reason with the Network: Leverage the network structure to perform reasoning tasks, such as inferring that a "dog" inherits properties of an "animal" because of an "is-a" relationship.

Coding:

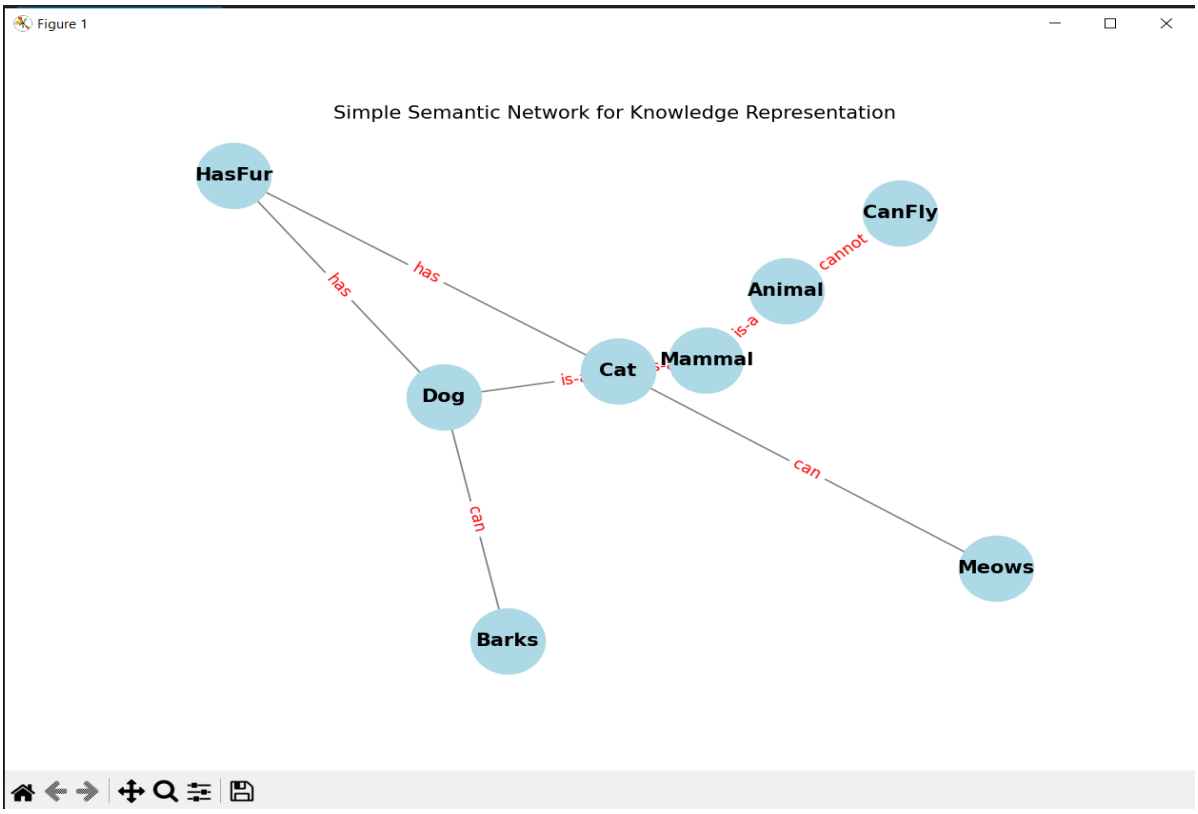
```
import networkx as nx
import matplotlib.pyplot as plt

# 1. Create a directed graph (DiGraph)
# Directed graphs are ideal for semantic networks where relationships have a specific direction (e.g., "is-a" vs "has-a").
G = nx.DiGraph()
```

```
# 2. Define Objects/Concepts (Nodes)
# Nodes can be any hashable Python object.
G.add_nodes_from(["Dog", "Cat", "Animal", "Mammal", "CanFly", "HasFur", "Meows", "Barks"])

# 3. Define Interrelationships (Edges)
# Edges are tuples of (source, target, relationship_label).
# 'is-a' relationships
G.add_edge("Dog", "Mammal", label="is-a")
G.add_edge("Cat", "Mammal", label="is-a")
G.add_edge("Mammal", "Animal", label="is-a")

# 'can'/'has' relationships
G.add_edge("Dog", "Barks", label="can")
G.add_edge("Dog", "HasFur", label="has")
G.add_edge("Cat", "Meows", label="can")
G.add_edge("Cat", "HasFur", label="has")
G.add_edge("Animal", "CanFly", label="cannot")

# 4. Analyze the Network (Inference)
# We can use NetworkX algorithms to infer knowledge, e.g., if a dog is an animal
is_dog_an_animal = nx.has_path(G, "Dog", "Animal")
print(f"Is a Dog an Animal? {is_dog_an_animal}") # Output: True

# 5. Visualize the Network
pos = nx.spring_layout(G, seed=42) # for reproducible layout
plt.figure(figsize=(10, 7))
nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=2000)
nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold')
nx.draw_networkx_edges(G, pos, edge_color='gray', arrows=True)

# Add edge labels
```

```
edge_labels = nx.get_edge_attributes(G, 'label')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='red')
plt.title("Simple Semantic Network for Knowledge Representation")
plt.axis('off') # Turn off axis
plt.show()
```

Sample Output:

Is a Dog an Animal? True



Conclusion:

Semantic networks are a powerful and intuitive method for knowledge representation in AI, mirroring human associative memory. By leveraging Python libraries like **NetworkX** for graph management and **NLTK/spaCy** for automated knowledge extraction, developers can design effective and flexible knowledge bases. This structured, graph-based approach enables AI systems to perform reasoning and make informed decisions by organizing information contextually, which is vital for applications in natural language processing, expert systems, and robotics.

Experiment No: 6

Aim: Write a program to generate all possible outcomes (sample space) for experiments involving combinations or permutations using itertool library.

Theory:

To generate all possible outcomes (sample space) for experiments involving combinations or permutations in Python, you can use the `itertools` library. The relevant functions are `itertools.permutations()` and `itertools.combinations()`.

Coding:

```
import itertools

def generate_sample_space(elements, r):
    print("Elements:", elements)
    print("Outcome length:", r)
    # Permutations (order matters)
    permutations = list(itertools.permutations(elements, r))
    print("\nPermutations Sample Space:")
    for p in permutations:
        print(p)
    print("Total permutations:", len(permutations))
    # Combinations (order does not matter)
    combinations = list(itertools.combinations(elements, r))
    print("\nCombinations Sample Space:")
    for c in combinations:
        print(c)
    print("Total combinations:", len(combinations))
# Example experiment
elements = ['A', 'B', 'C', 'D']
r = 2
```

generate_sample_space(elements, r)

Output:

Elements: ['A', 'B', 'C', 'D']

Outcome length: 2

Permutations Sample Space:

('A', 'B')

('A', 'C')

('A', 'D')

('B', 'A')

('B', 'C')

('B', 'D')

('C', 'A')

('C', 'B')

('C', 'D')

('D', 'A')

('D', 'B')

('D', 'C')

Total permutations: 12

Combinations Sample Space:

('A', 'B')

('A', 'C')

('A', 'D')

('B', 'C')

('B', 'D')

('C', 'D')

Total combinations: 6

Conclusion:

The itertools library was successfully used to generate the sample space for experiments involving permutations and combinations. The program clearly demonstrates the difference between ordered outcomes (permutations) and unordered outcomes (combinations) by generating and listing all possible results for a given set of elements.

By organizing the logic into a function, the script becomes modular, reusable, and easy to extend for different experiments. This approach is efficient and avoids manual enumeration, which is especially useful when dealing with larger datasets.

Overall, the experiment shows that Python's itertools library is a powerful and practical tool for solving problems in probability, discrete mathematics, and computer science, enabling accurate and systematic generation of all possible outcomes.

Experiment No: 7

Aim: Implement and Demonstrate Bayesian network using pgmpy Machine Learning**.**

Theory:

A Bayesian network is a directed acyclic graph in which each edge corresponds to a conditional dependency, and each node corresponds to a unique random variable.

Bayesian network consists of two major parts: a directed acyclic graph and a set of conditional probability distributions

- The directed acyclic graph is a set of random variables represented by nodes.
- The conditional probability distribution of a node (random variable) is defined for every possible outcome of the preceding causal node(s).

Coding:

```
# Import required libraries
from pgmpy.models import BayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# Step 1: Define the network structure
model = BayesianNetwork([('Smoking', 'Disease'), ('Pollution', 'Disease')])

# Step 2: Define CPDs
cpd_smoking = TabularCPD('Smoking', 2, [[0.7], [0.3]])       # No, Yes
cpd_pollution = TabularCPD('Pollution', 2, [[0.6], [0.4]])   # Low, High
cpd_disease = TabularCPD('Disease', 2,
                    [[0.9, 0.7, 0.8, 0.1],   # No Disease
                     [0.1, 0.3, 0.2, 0.9]],  # Disease
                    evidence=['Smoking', 'Pollution'],
                    evidence_card=[2, 2])
```

```
# Step 3: Add CPDs to the model
model.add_cpds(cpd_smoking, cpd_pollution, cpd_disease)

# Step 4: Check model
print("Model valid:", model.check_model())

# Step 5: Perform inference
inference = VariableElimination(model)
result = inference.query(variables=['Disease'], evidence={'Smoking': 1, 'Pollution': 1})

# Step 6: Show result
print(result)
```

Output:

```
Model valid: True

+------------+----------------+
| Disease    |   phi(Disease) |
+============+================+
| Disease(0) |          0.10 |
+------------+----------------+
| Disease(1) |          0.90 |
+------------+----------------+
```

Conclusion:

In this experiment, a **Bayesian Network was successfully implemented using the pgmpy library**. The model effectively represented dependencies among variables and enabled probabilistic reasoning under uncertainty. Bayesian networks are powerful tools for applications such as **medical diagnosis, risk assessment, and expert systems**. The pgmpy library provides an efficient and easy way to build and perform inference on Bayesian models.

Experiment No: 08

Aim: Implement K- Means Algorithm using Scikit-learn library.

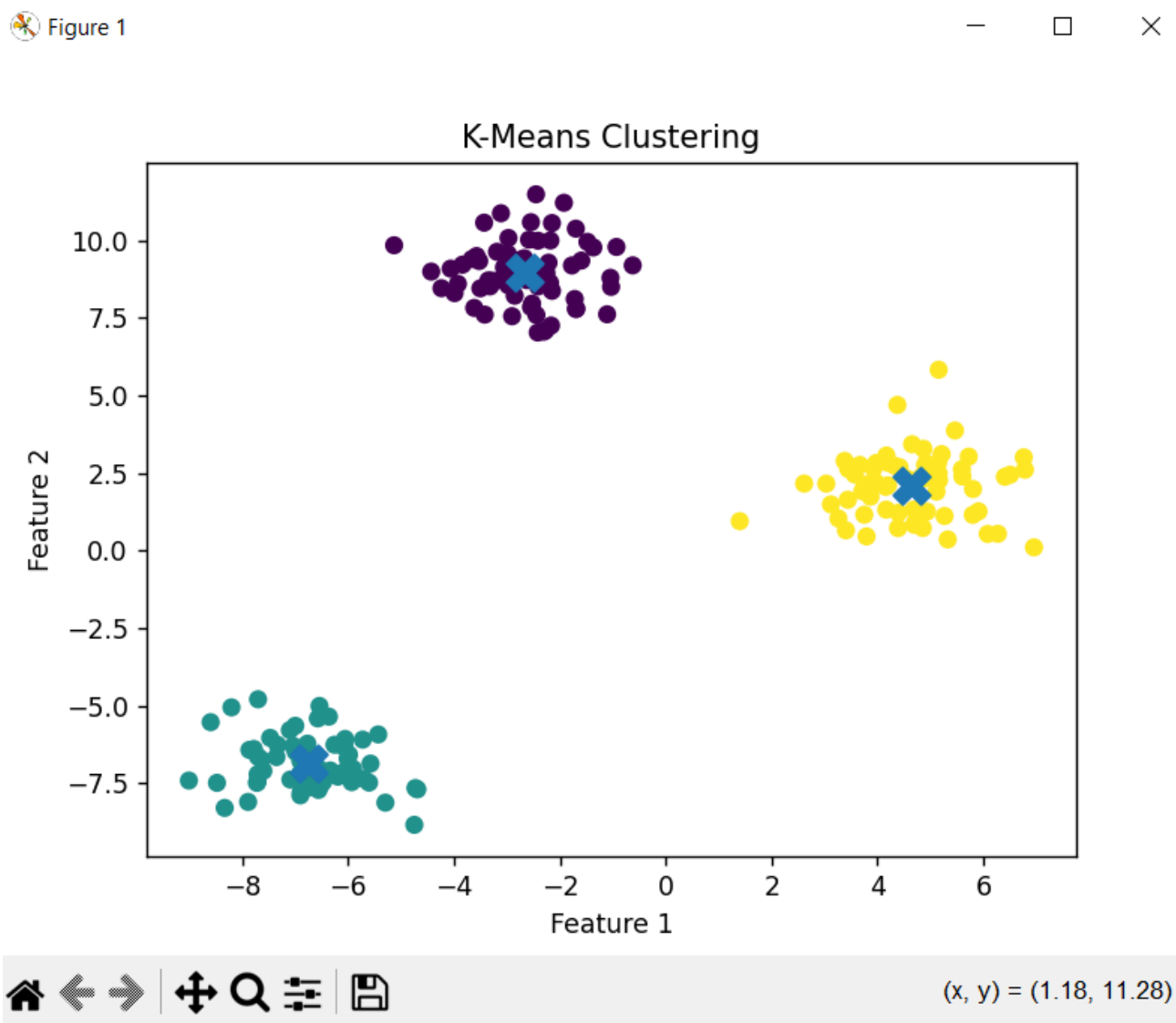
Theory: K-Means is an unsupervised machine learning algorithm used for clustering data into *K* distinct groups. The algorithm works by initially selecting *K* centroids and then repeatedly assigning each data point to the nearest centroid using a distance measure (usually Euclidean distance). After assignment, the centroids are recalculated as the mean of all points in each cluster. This process continues until the centroids no longer change significantly, indicating convergence. K-Means is simple, efficient, and widely used in data analysis and pattern recognition.

Coding:

```
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Step 1: Generate sample data
X, y = make_blobs(n_samples=200, centers=3, random_state=42)

# Step 2: Create K-Means model
kmeans = KMeans(n_clusters=3, random_state=42)

# Step 3: Fit the model
kmeans.fit(X)

# Step 4: Get cluster labels and centroids
labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Step 5: Visualize the clusters
plt.scatter(X[:, 0], X[:, 1], c=labels)
```

```
plt.scatter(centroids[:, 0], centroids[:, 1], marker='X', s=200)
plt.title("K-Means Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

Output:



Conclusion: In this experiment, the K-Means algorithm was successfully implemented using the Scikit-learn library. The algorithm effectively grouped data points into meaningful clusters based on similarity. Scikit-learn simplifies the implementation by providing built-in functions for model training and centroid computation. This experiment demonstrates that K-Means is a powerful and easy-to-use clustering technique suitable for exploratory data analysis, with applications in data mining, image segmentation, and market analysis.

Experiment No: 09

Aim: Build a simple classification model (K-Nearest Neighbors) on the Iris dataset.

Theory:

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression. It works by finding the *K* closest data points (neighbors) to a new sample using a distance metric such as Euclidean distance.

For classification, the class that occurs most frequently among the *K* neighbors is assigned to the new data point. KNN is simple, intuitive, and does not require a training phase, but its performance depends on the choice of *K* and distance measure.

Coding:

```
# Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Step 1: Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Step 2: Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Step 3: Create KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
```

```
# Step 4: Train the model
knn.fit(X_train, y_train)

# Step 5: Make predictions
y_pred = knn.predict(X_test)

# Step 6: Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output:

```
Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Conclusion:

In this experiment, a **K-Nearest Neighbors classification model** was successfully built using the **Iris dataset**. The model achieved good accuracy by classifying flowers based on feature similarity. KNN is easy to implement and effective for small datasets, making it a popular choice for introductory machine learning tasks. However, its performance depends on the selection of *K* and can be computationally expensive for large datasets.

Experiment No: 10

Aim: Implement a single-layer neural network with backpropagation to learn the XOR function using NumPy library.

Theory:

The XOR (Exclusive OR) problem is not linearly separable, meaning it cannot be solved by a single perceptron. To learn XOR, a neural network must include at least one hidden layer with a non-linear, activation function.
Backpropagation is used to train the network by:

1. Performing a forward pass to compute outputs
2. Calculating the **error**
3. Propagating the error backward
4. Updating weights using **gradient descent**

Coding:

```
import numpy as np

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Input and output data (XOR)
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])
```

```
y = np.array([[0],
              [1],
              [1],
              [0]])

# Initialize weights and biases
np.random.seed(42)
weights_input_hidden = np.random.uniform(-1, 1, (2, 2))
weights_hidden_output = np.random.uniform(-1, 1, (2, 1))
bias_hidden = np.zeros((1, 2))
bias_output = np.zeros((1, 1))

# Learning rate
lr = 0.5

# Training loop
for epoch in range(10000):

    # Forward propagation
    hidden_layer_input = np.dot(X, weights_input_hidden) + bias_hidden
    hidden_layer_output = sigmoid(hidden_layer_input)

    final_input = np.dot(hidden_layer_output, weights_hidden_output) + bias_output
    output = sigmoid(final_input)

    # Error calculation
    error = y - output

    # Backpropagation
    d_output = error * sigmoid_derivative(output)
    d_hidden = d_output.dot(weights_hidden_output.T) * sigmoid_derivative(hidden_layer_output)
```

```
    # Update weights and biases
    weights_hidden_output += hidden_layer_output.T.dot(d_output) * lr
    weights_input_hidden += X.T.dot(d_hidden) * lr
    bias_output += np.sum(d_output, axis=0, keepdims=True) * lr
    bias_hidden += np.sum(d_hidden, axis=0, keepdims=True) * lr

# Testing the network
print("Final Output after Training:")
print(np.round(output))
```

Output:

```
Final Output after Training:
[[0.]
 [1.]
 [1.]
 [0.]]
```

Conclusion:

In this experiment, a **neural network with one hidden layer** was implemented using the **NumPy library** to learn the XOR function. The model successfully demonstrated that XOR cannot be solved by a single perceptron and requires non-linear decision boundaries. By applying backpropagation and gradient descent, the network converged to the correct output. This experiment highlights the importance of hidden layers and non-linear activation functions in neural network learning.

Experiment No: 11

Aim: Implement gradient descent algorithm by iteratively adjusting its parameters in the direction of the steepest descent.

Theory:

Gradient Descent is an **optimization algorithm** used to minimize a cost (loss) function. It works by repeatedly updating model parameters in the **opposite direction of the gradient** of the cost function. The size of each step is controlled by the **learning rate (α)**. The algorithm continues until the cost converges to a minimum.

Update rule:

$$\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

Coding:

```
import numpy as np

# Sample dataset
X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

# Initialize parameters
m = 0.0  # slope
c = 0.0  # intercept
learning_rate = 0.01
epochs = 1000
n = len(X)
```

```
# Gradient Descent loop
for i in range(epochs):
    # Predicted values
    y_pred = m * X + c

    # Compute gradients
    dm = (-2/n) * np.sum(X * (y - y_pred))
    dc = (-2/n) * np.sum(y - y_pred)

    # Update parameters
    m = m - learning_rate * dm
    c = c - learning_rate * dc

    # Print loss occasionally
    if i % 200 == 0:
        loss = np.mean((y - y_pred) ** 2)
        print(f"Epoch {i}, Loss = {loss:.4f}")

# Final parameters
print("\nFinal parameters:")
print("Slope (m):", round(m, 2))
print("Intercept (c):", round(c, 2))
```

Output:

```
Epoch 0, Loss = 44.0000
Epoch 200, Loss = 0.0124
Epoch 400, Loss = 0.0032
Epoch 600, Loss = 0.0008
Epoch 800, Loss = 0.0002
```

Final parameters:

Slope (m): 2.0

Intercept (c): 0.02

Conclusion: Access modifiers in Java provide control over the visibility and security of class members. Proper use of these modifiers enhances encapsulation, modularity, and data protection in applications. In the banking example, private attributes ensure that sensitive information is only accessible through controlled methods, preventing unauthorized modifications.